When Should Random Testing Stop? Online Switching from Random Patterns to PODEM, Measured Against a Hindsight Oracle

Vidit Narayan Panigrahi and Varun V Nair
School of Electronics Engineering, Vellore Institute of Technology, Vellore, India
Course project, Testing of VLSI Circuits (BEVD309L)

***Abstract*—Hybrid test generators apply cheap random patterns first and hand the remaining faults to a deterministic ATPG algorithm. The switch point decides the cost, and the best point differs from circuit to circuit. We measure how close *online* switch rules, which decide from the coverage the run is producing, come to the best fixed switch point chosen in hindsight. A complete C++ flow (fault simulation, PODEM and a CDCL SAT fallback) resolves every fault, so all flows reach 100% fault efficiency and differ only in cost. We compare a plateau threshold, a Poisson test and a two-stage rule that also probes PODEM's yield with 19 fixed switch points. The oracle is defined separately for total work, for the work the switch point controls, for a combined cost that includes SAT effort, and for wall-clock time, and uncertainty is reported over seeds and over circuits. Settings were chosen on ISCAS-85 and then held fixed for six ISCAS-89 and ten ITC'99 circuits. On the work the switch controls, the plateau rule costs 1.031×, 1.029× and 1.096× the oracle in the three families (1.002–1.026× on combined cost), with a worst circuit at 1.52×. It is cheaper than every fixed switch count, including the best common count chosen in hindsight for each family, while the best count per circuit ranges from 192 to 32,768 patterns. The Poisson test makes the same decisions as the threshold, and probing PODEM before switching does not help. The contribution is the measurement rather than a new rule: it shows that a simple online threshold is enough and a fixed count is not.**

***Index Terms*—automatic test pattern generation, random pattern testing, PODEM, SAT-based ATPG, stuck-at faults, ISCAS and ITC'99 benchmarks**

I. Introduction

Random patterns graded by a fault simulator detect the easy majority of stuck-at faults for very little work per fault. Their yield then decays: the faults that remain are detected by few input combinations and random patterns almost never hit them. A deterministic algorithm such as PODEM [1] targets one fault at a time; it is far more expensive per fault, but reaches the hard faults and can prove a fault redundant. Practical flows therefore run random patterns first and give the rest to ATPG [2].

The switch point decides the cost. Too early, and PODEM spends effort on faults a few more random blocks would have caught. Too late, and the fault simulator keeps grading patterns that find almost nothing. Commercial tools stop at a fixed threshold, for example when a pattern detects less than 0.5% of the remaining faults [2]; coverage-prediction models estimate when random coverage saturates [3]–[5] but do not decide a switch.

Switching on a falling detection rate is therefore not new, and we do not claim the threshold rule as a contribution. What is missing is a measurement of how much such a rule loses against the best switch point, how that loss depends on the cost measure, and whether it carries over to circuits the rule was not tuned on. This paper asks: *how close does an online switch rule get to the best switch point chosen in hindsight, and does a statistically motivated rule do better than a simple threshold?* Our contributions are:

1) A complete, open test generator for combinational and full-scan circuits in which only the switch rule differs between flows, with a SAT fallback so that every fault ends detected or proved redundant. Flows can therefore be compared on cost alone, and no flow can look cheaper by giving up on hard faults.
2) An evaluation protocol: a per-circuit hindsight oracle over 19 fixed switch points, defined separately for four cost measures (total work, switch-dependent work, a combined cost that includes SAT effort, and wall-clock time); and confidence intervals both over seeds and over circuits.
3) A held-out test: all settings were fixed on ISCAS-85 before six ISCAS-89 and ten ITC'99 circuits were run.
4) The findings, including the negative ones: an online threshold beats every fixed switch count on every family; a fixed count tuned on one family does not transfer; a Poisson test reduces to the threshold; and probing PODEM before switching does not help.

II. Related Work

Threshold switching. The random-then-deterministic flow is standard; FastScan switches when the incremental detection rate of random patterns drops below a threshold [2]. Our plateau rule is of this kind.

Coverage prediction. STAFAN estimates detection probabilities from fault-free simulation [3]. Majumdar and Vrudhula derive the distribution of coverage and test length under random testing [4], and Cui, Seth and Mehta fit a coverage model to an initial run to decide when to stop fault simulation [5]. These predict saturation; they do not hand faults to ATPG or measure the cost of the hand-over.

Better random patterns and other hybrids. Weighted random generation tuned by a DNN-guided particle swarm raises cov-

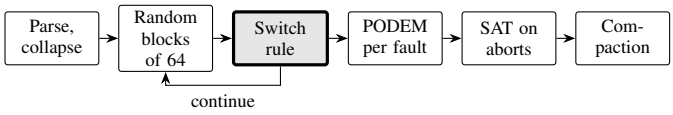


Fig. 1. The flow. Every method shares all stages; only the switch rule differs.

erage per pattern [6], and Wu et al. combine two scan-capture schemes inside deterministic ATPG [7]. Neither decides when random testing should stop.

SAT-based ATPG. Larrabee formulated test generation as Boolean satisfiability [8]. Industrial flows run structural ATPG under a backtrack limit and give its aborted faults to SAT [9], [10]; we use the same arrangement, so that the comparison of switch rules is not distorted by aborted faults.

III. Test Generation Framework

Fig. 1 shows the flow. The tool is about 3,900 lines of C++.

Circuits and faults. Netlists are read in `.bench` format and levelized with Kahn's algorithm. A flip-flop `q = DFF(d)` is read in full-scan form: q becomes a pseudo-primary input and d a pseudo-primary output. We target single stuck-at faults on every stem, fanout branch and branch into a primary output, collapsed by gate-level equivalence (24–51% fewer faults on ISCAS-85; c17 collapses to 22).

Fault simulation. Patterns are applied 64 at a time, one per bit of a machine word, using parallel-pattern single-fault propagation [11]: for each live fault only gates whose faulty value differs from the good value are evaluated, level by level, and a detected fault is dropped.

PODEM. PODEM [1] assigns primary inputs only and implies all other values in three-valued logic. Objectives and backtrace are guided by SCOAP controllability and observability [12]; a branch is pruned when the fault is not excited, the D-frontier is empty, or no X-path reaches an output. Each fault gets four attempts that share a budget of 1,024 backtracks (a short plain search, two short randomized ones, and a long plain one). Only an attempt that exhausts its search space may declare a fault redundant; any other failure is an abort. Every PODEM vector is fault-simulated before it is accepted.

SAT fallback. Each aborted fault is encoded as a miter: the good circuit, a faulty copy of the fault's fanout cone, and an OR of the output differences, in Tseitin clauses. It is solved by our own CDCL solver (two watched literals, first-UIP learning with clause minimization, VSIDS, phase saving, Luby restarts, LBD-based clause deletion), in the spirit of MiniSat [13], with a limit of 2×10^6 conflicts that no fault reached. A satisfying model is a test and is re-checked by fault simulation; an unsatisfiable miter proves the fault redundant.

Compaction. The final test set is fault-simulated in reverse order and a test is kept only if it is the first to detect some fault.

IV. Switch Rules

All rules observe the random phase block by block (64 patterns) after a warm-up of 128 patterns.

Fixed switch at N. Stop after N patterns, for 19 values of N from 64 to 32,768 (roughly $\sqrt{2}$ steps).

Plateau rule. Switch when the last six blocks together detected at most one new fault.

Stage 1: Poisson cost test. Random testing is worth continuing while its detection rate λ (faults per unit of work) exceeds the rate y at which PODEM would remove faults. A window of blocks grows until the expected number of detections under break-even, $\mu = y\,W$ for window work W, reaches 6. With k detections observed in the window, the null hypothesis $\lambda \geq y$ is rejected when

$$P(K \leq k \mid K \sim \mathrm{Poisson}(\mu)) = e^{-\mu} \sum_{i=0}^{k} \frac{\mu^i}{i!} < \alpha = 0.05. \tag{1}$$

Two consecutive rejections trigger the switch. Before PODEM has been measured, stage 1 uses a cost bound for y: a deterministic vector must be fault-simulated, which costs about as much as a 64-pattern block, so PODEM removes at most about one fault per block's worth of work. With $\mu = 6$, (1) gives 0.0174 for $k = 1$ and 0.0620 for $k = 2$, so a window is rejected exactly when it found $k \leq 1$ faults—the plateau rule's decision. The test therefore explains the threshold rather than improving on it; the two differ only in how the window is sized and in the confirmation step.

Stage 2: PODEM probe. After stage 1 rejects, the two-stage rule runs 16 PODEM calls on a random sample of the remaining faults to measure y on this circuit, and repeats the test against that yield; it re-probes when the targetable set halves. It switches at once if a probe (extended up to 48 calls) yields no test, or if fewer than μ targetable faults remain. This was the method proposed at the start of the project. *Stage 1* alone is the same rule with the probe turned off.

V. Evaluation Method

Cost measures. For each run we record four costs. (i) *Total work W*: gate evaluations, the same unit for fault simulation (including the good-circuit pass of each block) and PODEM implication. It is deterministic for a given seed and machine-independent. (ii) *Switch-dependent work W_{sw}*: W minus PODEM work on faults that end the run *proved* redundant (by PODEM, or by SAT after an abort). Proving redundancy costs 51–98% of total work on the nine ISCAS-85 circuits that have redundant faults; every flow pays it wherever it switches, so it dilutes the differences between rules. Because every flow resolves every fault, the set of redundant faults is a property of the circuit and identical for all flows, so the subtraction cannot favour any rule. (An earlier version subtracted work on every undetected fault, which let a flow lower its own score by aborting a detectable fault; we found and removed that bias.) (iii) *Combined cost* $W_{\mathrm{comb}} = W + c\,W_{\mathrm{SAT}}$, where W_{SAT} counts SAT propagation steps and c is the measured time of one propagation step divided by the measured time of one gate evaluation, over all runs of a benchmark family. This puts SAT effort, which (i) and (ii) leave out, into the

same unit. (iv) *Wall-clock time* T of the whole flow including compaction, on one core.

Oracle and regret. For a measure m and circuit C, the oracle is the fixed switch point $N_m^*(C)$ with the lowest mean of m over seeds. A flow's regret on m is its mean m divided by that of $N_m^*(C)$, aggregated by the geometric mean over circuits (c17 excluded). Each measure is scored against its own oracle; we also report the cross terms (a flow's cost on m against the point that is optimal for another measure), since the oracles need not coincide.

Uncertainty. We give two 95% percentile-bootstrap intervals (4,000 resamples, oracle recomputed inside each resample). *Seed-level* intervals resample seeds within each circuit and describe run-to-run noise on a fixed set of circuits. *Circuit-level* intervals first resample circuits with replacement and then seeds within each; they also cover the choice of circuits, and are much wider.

Benchmarks. All rule settings were chosen on the 11 ISCAS-85 circuits [14]. Six ISCAS-89 circuits [15] and ten ITC'99 circuits [16] (Table I) were then run with nothing changed. ISCAS circuits and b04–b14 use 10 seeds; b15, b17, b20, b21 and b22 use 3.

Verification. A built-in suite of 901 checks validates the SAT solver against exhaustive simulation (340 random circuits, about 70,000 faults), PODEM against exhaustive simulation and against SAT, and the full flow. An independent Python simulator with its own parser re-simulates the final test sets of all 17 ISCAS circuits and confirms the detected counts. The ISCAS-85 and ISCAS-89 sweeps were each run twice, with identical results in every deterministic column; b15, b17, b21 and b22 were run on a second machine, and the 70 runs of b15 and b21 that both machines completed agree in every deterministic column.

TABLE I
BENCHMARKS, RESOLVED FAULTS AND BEST FIXED SWITCH POINT

Circuit	Gates	FFs	Faults	Redundant	Best N
c432	160	–	524	4	768
c499	202	–	758	8	768
c880	383	–	942	0	1,024
c1355	546	–	1,574	8	2,048
c1908	880	–	1,879	9	3,072
c2670	1,269	–	2,747	117	1,024
c3540	1,669	–	3,428	137	4,096
c5315	2,307	–	5,350	59	3,072
c6288	2,416	–	7,744	34	192
c7552	3,513	–	7,550	131	2,048
s1423	657	74	1,515	14	2,048
s5378	2,779	179	4,551	40	4,096
s9234	5,597	211	6,927	452	3,072
s13207	7,951	638	9,815	151	16,384
s15850	9,772	534	11,725	389	6,144
s38417	22,179	1,636	31,180	165	16,384
b04	652	66	1,684	18	16,384
b05	927	34	2,470	542	1,024
b11	726	31	1,740	65	1,536
b12	944	121	2,872	0	1,536
b14	9,767	245	22,802	156	16,384
b20	19,682	490	45,459	319	32,768
b15	8,367	449	21,988	727	24,576
b21	20,027	490	46,154	378	16,384
b22	29,162	735	67,536	344	32,768
b17	30,777	1,415	76,625	1,958	1,536

Faults: collapsed. Every fault not listed as redundant is detected, by every flow except pure random, in every seed. Best N: oracle; it is the same for total work, switch-dependent work and combined cost on every circuit except b17 (combined: 768, a tie within 0.1%).

VI. RESULTS

A. Every fault is resolved

With the SAT fallback, every flow except pure random ends with each fault detected or proved redundant: fault efficiency is 100% on all 27 circuits, and the detected and redundant counts (Table I) are identical across flows and seeds. Almost every PODEM abort is a redundant fault that PODEM could not finish proving: on s9234, PODEM aborts on 210 faults and SAT proves all of them redundant; SAT occasionally finds a test PODEM missed (on c3540, c7552 and s38417). Without the fallback, some runs left a detectable fault undetected (3,290 instead of 3,291 on c3540), which is exactly the case that biased the earlier metric. Pure random testing proves nothing redundant and, within 32,768 patterns, falls short of the hybrids' detected count on c2670, c7552 and eight of the ten ITC'99 circuits in every seed; it detects 20,515 of b14's 22,646 detectable faults, 41,344 of b20's 45,140 and 18,195 of b15's 21,261.

B. Online rules against the oracle

Table II gives the main result. On ISCAS-85 all three online rules are within 0.4% of the oracle on total work and about 3% on switch-dependent work. Their pairwise ratios are indistinguishable from 1 (plateau ÷ two-stage: 0.998 [0.994, 1.004] on total work, 0.994 [0.968, 1.029] on switch-dependent work). Each beats the best common fixed point, @1024 (plateau ÷ @1024: 0.974 [0.967, 0.981] and 0.821 [0.775, 0.865]). PODEM alone costs 1.57× the oracle in total and 5.2× in switch-dependent work, so starting with random patterns is clearly right.

C. Held-out ISCAS-89 circuits

On ISCAS-89 the plateau rule and the Poisson test hold their ISCAS-85 numbers (1.029× and 1.028× on switch-dependent work) and remain tied (ratio 1.001 [0.992, 1.009]). The best fixed switch point moves: it is 2,048 to 16,384 patterns here against 192 to 4,096 on ISCAS-85, so the fixed count that did well on ISCAS-85 (@1024) costs 1.139× in total work, while @8192 improves from 1.082× to 1.023×. No single fixed count is good on both sets; the online rules are.

The two-stage rule switches late on two of the six circuits (Fig. 2). On s1423 the best N is 2,048; plateau switches at 1,722 patterns on average and the two-stage rule at 6,931, at 1.50× the oracle (2.3× on its worst seed). On s5378 it switches at 8,691 against a best N of 4,096. After stage 1 rejects, the probe sets a new break-even yield and random testing continues until that test also rejects twice. Over seeds, plateau ÷ two-stage is 0.933 [0.906, 0.960] on switch-dependent work, but the circuit-level interval is [0.825, 1.008]: the difference rests on these two circuits and is not established for the family.

TABLE II

COST RELATIVE TO THE HINDSIGHT ORACLE (GEOMETRIC MEAN OVER CIRCUITS; BELOW EACH VALUE, THE 95% SEED-LEVEL BOOTSTRAP INTERVAL)

Flow	ISCAS-85 (10 circuits, settings chosen here)		ISCAS-89 (6 circuits, held out)		ITC'99 (10 circuits, held out)	
	Total work	Switch-dependent	Total work	Switch-dependent	Total work	Switch-dependent
PODEM only	1.573 [1.553, 1.595]	5.160 [5.045, 5.378]	1.356 [1.352, 1.360]	2.193 [2.171, 2.219]	1.241 [1.237, 1.253]	1.664 [1.645, 1.714]
Fixed switch @128	1.273 [1.258, 1.290]	2.919 [2.833, 3.056]	1.282 [1.278, 1.288]	1.869 [1.838, 1.902]	1.152 [1.148, 1.162]	1.418 [1.400, 1.460]
Fixed switch @1024	1.029 [1.024, 1.037]	1.259 [1.204, 1.337]	1.139 [1.136, 1.143]	1.335 [1.318, 1.355]	1.060 [1.056, 1.071]	1.189 [1.171, 1.230]
Fixed switch @8192	1.082 [1.074, 1.092]	1.453 [1.442, 1.493]	1.023 [1.020, 1.026]	1.108 [1.095, 1.122]	1.064 [1.060, 1.075]	1.206 [1.192, 1.239]
Best common fixed N, in hindsight†	–	1.086	–	1.093	–	1.150
Plateau (6 blocks, $\leq$1 fault)	**1.002** [1.000, 1.008]	1.031 [1.015, 1.073]	1.003 [1.001, 1.007]	**1.029** [1.018, 1.044]	1.022 [1.019, 1.032]	1.096 [1.071, 1.141]
Stage 1 (Poisson test)	1.004 [1.002, 1.008]	**1.028** [1.014, 1.066]	**1.002** [1.000, 1.005]	**1.028** [1.017, 1.043]	**1.019** [1.016, 1.027]	**1.087** [1.068, 1.126]
Two-stage (Poisson + PODEM probe)	1.004 [1.002, 1.009]	1.035 [1.021, 1.070]	1.015 [1.011, 1.019]	1.104 [1.076, 1.133]	1.024 [1.019, 1.036]	1.111 [1.090, 1.150]

†The single fixed N with the lowest geometric-mean switch-dependent work over the family, chosen after seeing the results: 2,048 (ISCAS-85), 6,144 (ISCAS-89), 1,536 (ITC'99). No online rule was tuned on ISCAS-89 or ITC'99. b15, b17, b20, b21 and b22 use 3 seeds, all other circuits 10.

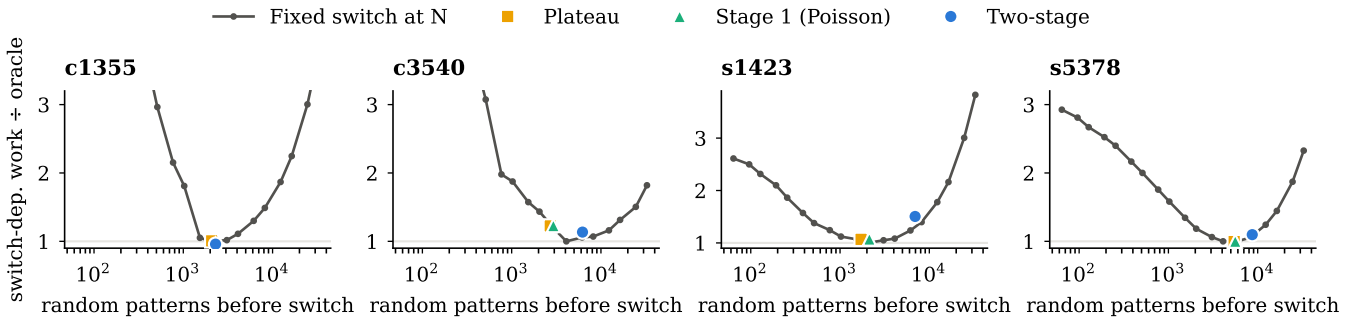


Fig. 2. Switch-dependent work against the switch point, divided by the best fixed point, mean over 10 seeds. Line: the 19 fixed switch points. Marks: mean switch point and cost of each online rule. c1355 and c3540 are from ISCAS-85, s1423 and s5378 from the held-out ISCAS-89 set.

D. Held-out ITC'99 circuits

The ITC'99 circuits are the hardest test. The best switch point ranges from 1,024 to 32,768 patterns. On b20 and b22 that is the top of the grid. We extended the grid with 49,152 and 65,536 patterns (random budget raised to match) on b20 and b22, and on b04 and b14, whose oracles at 16,384 had no point at four times the best N. The new points cost 1.02–1.03× (b20), 1.01–1.02× (b22), 1.64–2.03× (b04) and 1.13–1.22× (b14) of the existing oracle, so every oracle and every regret is unchanged; on b20 and b22 the cost curve is flat beyond about 24,576 patterns. The plateau rule costs 1.022× the oracle in total work and 1.096× in switch-dependent work—clearly more than the 3% of the ISCAS families. Its worst circuit is b04 (1.52×): the oracle switches at 16,384 patterns, but random blocks there find faults at about one per six blocks for a long time, and the rule stops at about 3,000. On b05 it switches late (2,560 against 1,024; 1.15×), and on b15 early (8,320 against 24,576; 1.13×). On the other seven circuits it is within 7%; on b17 it switches at 17,813 against 1,536, but that circuit's cost curve is nearly flat (1.055×).

The b04 failure follows from the assumption built into the threshold. Counting faults per block is the right signal when a PODEM call costs about as much as a random block, which is the bound stage 1 uses. On b04, a late random block simulates only the few dozen live faults left and costs about 940 gate evaluations, while each PODEM call that the extra random patterns make unnecessary costs about 16,300; break-even is therefore about one fault per 17 late blocks, not one per six. The two-stage rule measures PODEM's cost and switches later (12,928 patterns on average), but its 21 probe calls cost more than the later switch saves.

A longer window is not a fix. Checked after the fact on the six smaller ITC'99 circuits, and not used to choose any setting, a window of 12 blocks brings b04 to 1.15× and one of 24 blocks to 1.06×, but b05 rises to 1.40× and 1.86× and b12 to 1.12× and 1.29×. Over those six circuits the regret is 1.117× and 1.215× against 1.120× for the default, and both longer windows are worse on ISCAS-89 (1.068× and 1.195× against 1.029×). No block count suits every circuit, because the count cannot see what PODEM costs on the circuit at hand; a rule that does would need a cheaper estimate of that cost than our probe.

The rule still beats every fixed count: @1024 costs 1.189×, @8192 1.206×, and even the best common count chosen in hindsight for this family (1,536) costs 1.150×. Over seeds, plateau ÷ @1024 is 0.921 [0.901, 0.939]. The Poisson test

TABLE III
COST MEASURES, ORACLE CHOICE AND CIRCUIT-LEVEL UNCERTAINTY

	ISCAS-85	ISCAS-89	ITC'99
SAT share of run time (all flows)	9.8–11.4%	3.5–3.9%	39–40%‡
c (gate evaluations per SAT step)	4.9	4.7	1.8‡
Circuits where the oracle matches the total-work oracle			
switch-dependent	10/10	6/6	10/10
combined	10/10	6/6	9/10
wall-clock	1/10	2/6	1/10
Regret on combined cost (own oracle)			
Plateau	1.002	1.003	1.026
Stage 1	1.004	1.002	1.018
Two-stage	1.004	1.015	1.028
Fixed @1024	1.028	1.135	1.055
Wall-clock time against the work oracle's N			
Plateau	1.024	0.995	1.011
Fixed @1024	1.035	1.043	1.043
PODEM only	1.392	1.290	1.089
Wall-clock time against the wall-clock oracle			
Plateau	1.072	1.009	1.156‡
Fixed @1024	1.083	1.057	1.192‡
Switch-dependent work, circuit-level 95% interval			
Plateau	[1.01, 1.10]	[1.00, 1.06]	[1.04, 1.21]
Two-stage	[1.01, 1.09]	[1.01, 1.26]	[1.04, 1.24]
Fixed @1024	[1.08, 1.52]	[1.18, 1.51]	[1.08, 1.39]
Plateau ÷ two-stage	[0.95, 1.04]	[0.83, 1.01]	[0.95, 1.02]
Plateau ÷ @1024	[0.70, 0.95]	[0.67, 0.90]	[0.84, 1.01]

‡ITC'99 timings were taken on two machines, each shared with other sweeps, and are not reliable; see text.

again matches the threshold (ratio 1.008 [0.993, 1.024]). The two-stage rule is not distinguishable from the plateau rule here (0.985 [0.964, 1.009]), and stage 1 alone is cheaper than the two-stage rule (0.978 [0.963, 0.994]): the probe never helps.

E. Does the cost measure change the answer?

SAT effort. Total and switch-dependent work leave out SAT effort, so a flow that sent more faults to SAT could look cheaper than it is. Table III checks this. SAT takes about 11% of run time on ISCAS-85, 4% on ISCAS-89 and about 40% on ITC'99, and the number of SAT calls and propagation steps is nearly the same for every flow: SAT work is spent on faults that are hard for PODEM, mostly redundant ones, and the switch rule hardly changes which those are. The per-step weight c is lower on ITC'99 because its SAT calls are over 100 times longer, so the fixed cost of building each miter is spread over more propagation steps. On the combined cost, the regrets move by at most 0.01, and the ranking is unchanged in every family.

Because c is measured and differs between families, we recomputed the combined cost for every c from 0 to 20. On both ISCAS families the three work-based oracles choose the same N for every c, the rules' regrets change by at most 0.001, and their lead over every fixed point remains. On ITC'99, b17 and b15 have flat, SAT-heavy cost curves, so their combined-cost oracles move once SAT counts (b17 from 1,536 to 768 patterns for $c \geq 1.5$, b15 above $c = 6$), each by less than 0.3% in cost. The plateau rule happens to send slightly more work to SAT, so its combined regret rises from 1.022 at $c = 0$ to 1.037 at $c = 20$; above $c \approx 8$ the best common fixed N chosen in hindsight (1,536 patterns, 1.029–1.032×) becomes marginally cheaper than the plateau rule, but not than stage 1 (1.018×). At the measured $c = 1.8$ the plateau rule is well ahead.

Oracle definitions. The total-work, switch-dependent and combined-cost oracles select the same N on 25 of the 26 circuits; on b17 the combined-cost oracle picks 768 instead of 1,536 patterns, 0.06% cheaper, which is a tie. For the three work measures the regret is therefore unambiguous. The wall-clock oracle differs on most circuits. Timing is noisy (median seed-to-seed variation 7–10% on the millisecond-scale ISCAS-85 runs), and the minimum of 19 noisy means is biased low. Measured in time against the N the work oracle chooses, the plateau rule is within 2.4% in every family, while PODEM alone is 1.09–1.39× slower. The ITC'99 times were taken while other sweeps ran on the same machine (a two-core machine, and for b15, b17, b21 and b22 a second machine running four sweeps at once); under that load the wall-clock oracle sits 14–19% below every online rule and every listed fixed point, and picks switch points as low as 64 patterns, and we do not draw conclusions from those times. Peak memory is 11–20 MB on the ISCAS circuits, 14 MB on b14 and 32 MB on b20, and does not depend on the rule.

F. Uncertainty over circuits

The seed-level intervals in Table II are narrow because they hold the circuits fixed. Resampling circuits as well (Table III, bottom) widens them several-fold: the plateau rule's switch-dependent regret is [1.01, 1.10] on ISCAS-85 and [1.04, 1.21] on ITC'99. The advantage over @1024 is clear at circuit level on both ISCAS families but not established on ITC'99 ([0.84, 1.01]), and plateau and two-stage cannot be told apart at circuit level in any family. With six and ten circuits in the held-out families, the numbers describe these circuits; they do not bound the regret on unseen designs.

G. Early switching costs more than late switching

The fixed-N curves are asymmetric on the ISCAS families (Fig. 2). On ISCAS-85, switching at half the best N costs 1.03–1.83× the optimum in switch-dependent work and at a quarter up to 3.90×, while switching at twice the best N costs 1.00–1.35× and at four times up to 2.07×. On ISCAS-89 the curves are milder (quarter: up to 1.58×; four times: up to 1.44×). On ITC'99 the asymmetry does not hold: a quarter of the best N costs at most 1.40×, while four times the best N costs up to 1.36× on the small circuits and 2.03× on b04 (from the extended grid). On the ISCAS families the online rules switch at 0.7–2.9× the best N, the safe side. On ITC'99 the plateau rule switches between 0.18× (b04, its worst case) and 11.6× (b17, where the curve is nearly flat) of it.

H. Robustness and other costs

Settings. Varying each rule's own settings (18 variants: window, threshold, α, confirmations, probe size, warm-up) keeps every variant within about 1% of the oracle on total work on ISCAS-85.

Cost of PODEM. Repeating the ISCAS-85 sweeps with PODEM work weighted by 0.25, 4 and 16 (5 seeds, each

with its own oracle) keeps all online rules ahead of every fixed switch point; their gap to the oracle grows to 1.075–1.088× in total work at weight 16.

PODEM budget. Because SAT resolves every abort, the backtrack limit trades PODEM work for SAT work. Lowering it from 1,024 to 32 cuts the two-stage rule's total ISCAS-85 work from 76.5 M to 10.5 M gate evaluations, sends 297 faults to SAT instead of 215, and keeps fault efficiency at 100%; the ranking of the rules is unchanged.

Test size and run time. After compaction every hybrid rule produces 0.97× the tests of PODEM alone on ISCAS-85, 1.00–1.01× on ISCAS-89 and 1.01–1.03× on ITC'99, so the rule hardly changes test size; the plateau rule takes 0.70× the time of PODEM alone on ISCAS-85 and 0.76× on ISCAS-89. A run takes under 2 s on ISCAS, 9 s on b14 and 31 s on b20.

VII. Discussion and Limitations

What the results support. Choosing the switch point online is worth doing. Averaged over each family, a one-line threshold beats every fixed switch count in all three, including a count tuned in hindsight for each family (at circuit level the ITC'99 margin over @1024 is not significant), while the best count per circuit spans more than two orders of magnitude. The margin to the oracle depends on the family: about 3% of switch-dependent work on ISCAS, 10% on ITC'99. Which online rule is used matters less than expected. The Poisson test does not beat the threshold; it explains it, since at $\mu = 6$ and $\alpha = 0.05$ the two make the same decision. As a switch rule it adds machinery without a measurable gain, and we would use the threshold; its value is that it makes the threshold's hidden assumption explicit—that a PODEM call costs about one random block—which is exactly what fails on b04. Measuring PODEM's yield before switching sounded like the principled refinement, but it never reduced cost, and on two ISCAS-89 circuits it delayed the switch.

Novelty. The plateau rule is an incremental-coverage threshold of the kind industrial tools already use [2]. The contribution is the measurement around it: a fault-complete flow in which no rule can gain by abandoning faults, an oracle over a grid of switch points defined separately for each cost measure, a metric that removes the work no rule can avoid, intervals over circuits as well as seeds, and a held-out evaluation on two further families.

Limitations. The held-out families have six and ten circuits, and the circuit-level intervals are wide; industrial designs have not been run, and five ITC'99 circuits have only 3 seeds. Sequential circuits are handled only through full scan. The ITC'99 wall-clock times are unreliable. The combined cost depends on a measured constant c that varies between families (1.8–4.9); the ISCAS conclusions hold for any c from 0 to 20, but on ITC'99 a hindsight-tuned fixed count overtakes the plateau rule above $c \approx 8$. Our PODEM is basic (no static learning or unique sensitization); a stronger engine would shift the best switch points but not the comparison, since every flow shares it. The oracle is an evaluation baseline, not an implementable algorithm; being close to it measures the quality of the decision, not global optimality.

VIII. Conclusion

A one-line online rule—switch when six blocks of 64 random patterns find at most one new fault—costs 1.03× the best fixed switch point in switch-dependent work on ISCAS-85 and on held-out ISCAS-89 circuits, and 1.10× on held-out ITC'99 circuits, with every fault resolved by a SAT fallback. Counting SAT effort does not change the result, and the three work-based oracles agree on every circuit but one, where they tie. On average the rule beats every fixed switch count in every family, and a fixed count tuned on one family does not transfer to another. A Poisson test makes the same decisions as the threshold, and probing PODEM before switching does not help. Code, data and every number in this paper regenerate from https://github.com/viditezz/vlsi-fault-testing.

References

[1] P. Goel, "An implicit enumeration algorithm to generate tests for combinational logic circuits," *IEEE Trans. Computers*, vol. C-30, no. 3, pp. 215–222, Mar. 1981.

[2] Auburn University, "Automatic test pattern generation," ELEC 5250/6250 lecture slides (FastScan ATPG method). [Online]. Available: https://www.eng.auburn.edu/~nelson/courses/elec5250_6250/slides/Test_ATPG.pdf

[3] S. K. Jain and V. D. Agrawal, "STAFAN: An alternative to fault simulation," in *Proc. 21st Design Automation Conf.*, 1984.

[4] A. Majumdar and S. B. K. Vrudhula, "Fault coverage and test length estimation for random pattern testing," *IEEE Trans. Computers*, vol. 44, no. 2, Feb. 1995.

[5] H. Cui, S. C. Seth, and S. K. Mehta, "Modeling fault coverage of random test patterns," *J. Electronic Testing: Theory and Applications*, vol. 19, pp. 271–284, 2003.

[6] H. Maity, K. Khatua, S. Chattopadhyay, I. Sengupta, G. Patankar, and P. Bhattacharya, "Fault coverage enhancement via weighted random pattern generation in BIST using a DNN-driven-PSO approach," in *Proc. Int. Conf. Information Technology (ICIT)*, 2019.

[7] S. Wu, L.-T. Wang, Z. Jiang, J. Song, B. Sheu, X. Wen, M. S. Hsiao, J. C.-M. Li, J.-L. Huang, and R. Apte, "On optimizing fault coverage, pattern count, and ATPG run time using a hybrid single-capture scheme for testing scan designs," in *Proc. IEEE Int. Symp. Defect and Fault Tolerance of VLSI Systems (DFT)*, 2008.

[8] T. Larrabee, "Test pattern generation using Boolean satisfiability," *IEEE Trans. Computer-Aided Design*, vol. 11, no. 1, pp. 4–15, Jan. 1992.

[9] J. Huang, H.-L. Zhen, N. Wang, M. Yuan, H. Mao, Y. Huang, and J. Tao, "Accelerate SAT-based ATPG via preprocessing and new conflict management heuristics," in *Proc. 27th Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 2022, pp. 365–370.

[10] H.-L. Zhen, N. Wang, J. Huang, X. Huang, M. Yuan, and Y. Huang, "Conflict-driven structural learning towards higher coverage rate in ATPG," arXiv:2303.02290, 2023.

[11] J. A. Waicukauski, E. B. Eichelberger, D. O. Forlenza, E. Lindbloom, and T. McCarthy, "Fault simulation for structured VLSI," *VLSI Systems Design*, Dec. 1985.

[12] L. H. Goldstein and E. L. Thigpen, "SCOAP: Sandia controllability/observability analysis program," in *Proc. 17th Design Automation Conf.*, 1980.

[13] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT 2003)*, LNCS 2919, 2004.

[14] F. Brglez and H. Fujiwara, "A neutral netlist of 10 combinational benchmark circuits and a target translator in Fortran," in *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS)*, 1985.

[15] F. Brglez, D. Bryan, and K. Kozminski, "Combinational profiles of sequential benchmark circuits," in *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS)*, May 1989, pp. 1929–1934.

[16] F. Corno, M. Sonza Reorda, and G. Squillero, "RT-level ITC'99 benchmarks and first ATPG results," *IEEE Design & Test of Computers*, vol. 17, no. 3, pp. 44–53, Jul.–Sep. 2000.